

```

1 #=====
2 # MIT 6.046J 代码精读 • PS1 - 树最大权独立集 DP
3 # 源文件: restaurant_location.py (来自 GitHub mit-6046)
4 #=====
5
6 """PS1 Problem 1-1: Restaurant Location (Maximum-Weight Independent Set).
7
8 Drunken Donuts wants to place restaurants on vertices of a street graph to
9 maximize total profit, subject to: no two chosen vertices may be adjacent.
10 This is exactly the Maximum-Weight Independent Set (MWIS) problem.
11
12 - Parts (a)-(c): the graph is a tree. MWIS on a tree is solvable in  $O(V)$  time
13   by dynamic programming over the rooted tree.
14 - Part (a): a greedy "take the globally heaviest, delete its neighbours, repeat"
15   rule is not optimal; ``greedy_counterexample`` builds and verifies a concrete
16   path on which greedy loses.
17 - Part (d): general graphs  $\rightarrow$  MWIS is NP-hard (see PS8), so the best correct
18   general algorithm is exponential; ``mwis_general_bruteforce`` is a correct
19   exact solver used here to check the tree DP.
20
21 Everything here runs and is checked against brute force in ``test``.
22 """
23 from __future__ import annotations
24
25 import itertools
26 from typing import Dict, Iterable, List, Set, Tuple
27
28
29 Graph = Dict[int, Set[int]]
30
31
32 def make_graph(n: int, edges: Iterable[Tuple[int, int]]) -> Graph:
33     g: Graph = {u: set() for u in range(n)}
34     for u, v in edges:
35         g[u].add(v)
36         g[v].add(u)
37     return g
38
39
40 # -----
41 # Tree DP (parts b, c)
42 # -----
43 def mwis_tree(profit: List[int], tree: Graph, root: int = 0) -> Tuple[int, Set[int]]:
44     """Maximum-weight independent set of a tree,  $O(V)$  time.
45
46     DP: for each vertex u rooted at ``root`` define
47         inc[u] = best total using u (so no child of u is used)
48         exc[u] = best total not using u (each child free to be used or not)
49     with recurrences
50         inc[u] = profit[u] + sum(exc[c] for children c)
51         exc[u] = sum(max(inc[c], exc[c]) for children c)
52     The answer is max(inc[root], exc[root]). We reconstruct the set by a
53     second top-down pass. Implemented iteratively (post-order) to avoid
54     recursion-depth limits on long paths.
55     """
56     n = len(profit)
57     parent = [-1] * n
58     order: List[int] = []
59     seen = [False] * n
60     stack = [root]
61     seen[root] = True
62     while stack: # iterative DFS producing a pre-order; reverse => post-order
63         u = stack.pop()
64         order.append(u)
65         for c in tree[u]:
66             if not seen[c]:
67                 seen[c] = True
68                 parent[c] = u
69                 stack.append(c)
70
71     inc = [0] * n
72     exc = [0] * n
73     children: Dict[int, List[int]] = {u: [] for u in range(n)}
74     for u in order:
75         if parent[u] != -1:
76             children[parent[u]].append(u)
77
78     for u in reversed(order): # post-order: children processed before u
79         inc[u] = profit[u]
80         exc[u] = 0
81         for c in children[u]:
82             inc[u] += exc[c]
83             exc[u] += max(inc[c], exc[c])
84
85     # reconstruct

```

```

86     chosen: Set[int] = set()
87     take_root = inc[root] >= exc[root]
88     decide = [(root, take_root)]
89     while decide:
90         u, take = decide.pop()
91         if take:
92             chosen.add(u)
93             for c in children[u]:
94                 decide.append((c, False)) # cannot take a child of a taken node
95         else:
96             for c in children[u]:
97                 decide.append((c, inc[c] >= exc[c]))
98     return max(inc[root], exc[root]), chosen
99
100
101 def mmwis_tree_maxcount(tree: Graph, root: int = 0) -> Tuple[int, Set[int]]:
102     """Part (c): all sites equal -> maximise the *number* of chosen vertices.
103
104     This is just MWIS with every profit == 1 (equivalently maximum independent
105     set of a tree). The classic greedy for this special case -- repeatedly take
106     a leaf, delete it and its neighbour -- is optimal; here we simply reuse the
107     unit-weight DP, which is provably optimal and just as fast.
108     """
109     n = len(tree)
110     return mmwis_tree([1] * n, tree, root)
111
112
113 # -----
114 # General graph exact solver (part d, and used as an oracle)
115 # -----
116 def mmwis_general_bruteforce(profit: List[int], graph: Graph) -> Tuple[int, Set[int]]:
117     """Exact MWIS on an arbitrary graph by trying every independent set.
118
119     Exponential (correct) -- used to certify the tree DP and to stand in for
120     part (d), where no poly-time algorithm is expected (DONUT is NP-hard, PS8).
121     """
122     n = len(profit)
123     best_val = -1
124     best_set: Set[int] = set()
125     verts = list(range(n))
126     for r in range(n + 1):
127         for combo in itertools.combinations(verts, r):
128             s = set(combo)
129             if all(v not in graph[u] for u in s for v in graph[u] if v in s):
130                 # independence check
131                 ok = True
132                 for u in s:
133                     if graph[u] & s:
134                         ok = False
135                         break
136                 if ok:
137                     val = sum(profit[u] for u in s)
138                     if val > best_val:
139                         best_val, best_set = val, s
140     return best_val, best_set
141
142
143 # -----
144 # Part (a): greedy counterexample
145 # -----
146 def greedy_max_profit(profit: List[int], graph: Graph) -> Tuple[int, Set[int]]:
147     """The (sub-optimal) greedy of part (a): repeatedly take the globally
148     highest-profit remaining vertex, delete it and its neighbours."""
149     alive = set(range(len(profit)))
150     chosen: Set[int] = set()
151     while alive:
152         u = max(alive, key=lambda x: (profit[x], -x))
153         chosen.add(u)
154         alive.discard(u)
155         alive -= graph[u]
156     return sum(profit[u] for u in chosen), chosen
157
158
159 def greedy_counterexample() -> dict:
160     """A concrete tree on which the greedy of part (a) is *not* optimal.
161
162     Path 0 - 1 - 2 with profits [2, 3, 2]. Greedy grabs vertex 1 (profit 3)
163     and then can take nothing else -> 3. Optimum takes {0, 2} -> 4.
164     """
165     profit = [2, 3, 2]
166     tree = make_graph(3, [(0, 1), (1, 2)])
167     g_val, g_set = greedy_max_profit(profit, tree)
168     opt_val, opt_set = mmwis_tree(profit, tree)
169     return {
170         "profit": profit,
171         "edges": [(0, 1), (1, 2)],

```

```
172     "greedy_value": g_val,  
173     "greedy_set": sorted(g_set),  
174     "optimal_value": opt_val,  
175     "optimal_set": sorted(opt_set),  
176     "greedy_is_suboptimal": g_val < opt_val,  
177 }  
178
```